

PYTHON PROGRAMMING FOR EEE STUDENTS

Session 01: Core Fundamentals

Md. Owaliur Rahman

Lecturer, Department of CSE, Presidency University

`hellofihad@gmail.com`

WHAT IS PYTHON?

Python is an **interpreted, high-level** language that focuses on code readability. It allows EEE engineers to automate tasks without complex syntax.

★ Simple English-like structure.

⟨/⟩ Versatile for both small scripts and large systems.

§ Massive open-source library ecosystem.

WHY PYTHON FOR EEE?



Hardware Control

Interfacing with microcontrollers like Raspberry Pi and ESP32.



Signal Processing

Analyzing waveforms and sensor data using SciPy/NumPy.



Power Analysis

Simulating load flow and transient response in power systems.

BROAD INDUSTRY USAGE

Machine Learning

Autonomous drones and smart grid prediction.

Cybersecurity

Network sniffing and encryption protocols.

Data Science

Statistical analysis of smart-meter data.

Robotics

Motion planning and sensor fusion logic.

SETUP & INSTALLATION

- ▶ **Download:** Python 3.x from `python.org`.
- ▶ **Crucial:** Tick “Add Python to PATH” during install.
- ▶ **Editor:** Use Visual Studio Code (VS Code).

The IDLE shell (or VS Code’s integrated terminal) lets you run code line-by-line and inspect variables instantly with `globals()`.

RUNNING PYTHON

1. Open Terminal / CMD.
2. Type python to open the interpreter.
3. To run a saved file:

```
python script_name.py
```

THE PRINT() FUNCTION

Display messages to the console. Text must be inside quotes.

```
print("Welcome to Presidency University")  
print('I am learning Python')  
print("Python is easy to read")
```

```
Welcome to Presidency University  
I am learning Python  
Python is easy to read
```

NEWLINES & BLANK LINES

Empty `print()` adds vertical space.

```
print("Line 1")  
print() # Blank  
print("Line 3")
```

Escape characters:

```
print("Top\nBottom")
```

- `\n` – newline
- `\t` – tab
- `\\` – backslash
- `\'` – single quote
- `\"` – double quote

THE “SEP” PARAMETER

Changes the separator between multiple items (default is space).

```
print("Voltage", 220, sep=": ")  
print("2026", "06", "14", sep="-")  
print("Python", "Java", "C++", sep=", ")
```

```
Voltage: 220  
2026-06-14  
Python, Java, C++
```

THE “END” PARAMETER

Controls what happens at the end of the line (default is newline).

```
print("Loading", end="...")  
print("Complete")  
print("A", end=",")  
print("B", end=",")  
print("C")
```

```
Loading...Complete  
A,B,C
```

STRING ADDITION

Join (concatenate) strings using the + operator.

```
print("Hello" + " " + "World")  
print("Python" + " is awesome")  
print("Error code: " + "404")
```

```
Hello World  
Python is awesome  
Error code: 404
```

Note: You cannot directly add a string and a number.

```
print("Value: " + 42) → TypeError
```

Use `str()` to convert: `print("Value: " + str(42))`

STRING MULTIPLICATION

Repeat a string multiple times using the * operator.

```
print("-" * 50) # Creating a divider  
print("a" * 10)  
print("Circuit! " * 3)
```

```
-----  
aaaaaaaaaa  
Circuit!  Circuit!  Circuit!
```

CALCULATIONS INSIDE PRINT()

Python calculates before displaying. No quotes needed for numbers.

```
print(10 + 5)
print("Resistance:", 50 + 25, "Ohms")
print("2 + 3 =", 2 + 3)
```

```
15
Resistance: 75 Ohms
2 + 3 = 5
```

PRACTICE: PRINT() CHALLENGE

What is the **exact output** of the following code?

```
print("Error", 404, sep=":", end="\n" + "-" * 10 + "\n")
print("Retry", end=" in ")
print(3 * 2, "seconds", sep="-")
print()
print("Status", "OK", sep=" ", end="!")
```

Write the complete output **exactly** as it would appear (including blank lines and spacing).

VARIABLES: DATA CONTAINERS

Store values to use them later.

name = value

```
voltage = 12
current = 2
power = voltage * current
print(power)
```

■ Stores text, numbers, or logic.

⇔ Values can be updated.

NAMING RULES

Rule	Correct	Incorrect
Start with a letter or _	v1	1v
No spaces	total_marks	total marks
Case sensitive	Age $\neq$ age	–

THE CORE DATA TYPES

“ ”

String (str)

Text data
wrapped in
quotes. Example:
"Ehsan"

#

Integer (int)

Whole numbers
without
decimals.
Example: 220



Float (float)

Decimal numbers
for precision.
Example: 3.14



Boolean (bool)

Logic states: True
or False.

DEEP DIVE: STRINGS (STR)

Quotes & Characters

Both ' ' and " " work equally. Even a single letter is a string.

```
char = 'a'  
word = "Circuit"  
print(type(char))
```

The String Trap

Numbers in quotes are treated as text, not math!

```
x = "10"  
y = "20"  
print(x + y) # Result: 1020
```

INTEGERS VS. FLOATS

Integer (int): Counting objects, years, indices. No decimals.

```
students = 50  
year = 2024
```

Float (float): Measurement, voltage, current, division results.

```
volts = 220.5  
pi = 3.14159
```

Note: 5.0 is a float, while 5 is an integer.

IDENTIFYING TYPES: TYPE()

Use the `type()` function to check the data type of any variable or value.

```
print(type(50))  
print(type(3.14))  
print(type("Python"))
```

name	keyword	description
string	<code>str</code>	sequence of characters
integer	<code>int</code>	whole number
float	<code>float</code>	decimal number
boolean	<code>bool</code>	True/False

VARIABLES IN ENGINEERING

Meaningful names prevent errors in calculations.

```
v = 220 # Voltage  
i = 5 # Current  
p = v * i  
print(f"Power: {p}W")
```

PROFESSIONAL OUTPUT: F-STRINGS

Embed variables directly inside strings using `f"{var}"`.

```
name = "Rahat"  
score = 95  
print(f"Student {name} scored {score} marks")  
print(f"2 + 3 = {2 + 3}")
```

F-STRINGS: FORMAT SPECIFIERS

Control precision: `:.2f`.

```
current = 2.45678
print(f"Current: {current:.2f} A")
print(f"Current: {current:.3f} A")
```

```
Current: 2.46 A
Current: 2.457 A
```

Spec	Meaning	Ex.
<code>:.2f</code>	2 decimals	2.46
<code>:.0f</code>	No decimals	2
<code>:.2e</code>	Scientific	2.46e+00
<code>:.1%</code>	Percentage	245.7%
<code>:,</code>	Thousands	1,000

THE INPUT() FUNCTION

Asking the user for data. Execution pauses until the user hits Enter.

```
name = input("Enter your name: ")  
print(f"Hello, {name}!")  
  
city = input("Enter your city: ")  
print(f"You live in {city}")
```

THE INPUT TRAP

`input()` always returns a **STRING (str)**. Math won't work on it immediately!

```
a = input("Enter 10: ")  
b = input("Enter 20: ")  
print(a + b) # Result: 1020
```

TYPE CONVERSION (CASTING)

Convert input text to numbers using `int()` or `float()`.

```
age = int(input("Enter your age: "))  
height = float(input("Enter height: "))  
  
print(f"Next year you will be {age + 1}")
```

Note: `int()` for whole numbers, `float()` for decimals.

CONCATENATION VS. ADDITION

Addition (Math)

Numbers + Numbers = Sum

```
a = 10  
b = 20  
print(a + b) # Output: 30
```

Concatenation (Join)

Strings + Strings = Joined Text

```
a = "10"  
b = "20"  
print(a + b) # Output: 1020
```

THE MODULUS OPERATOR (%)

The % operator gives the **remainder** of a division.

```
print(5 % 3) # Remainder is 2
print(21 % 3) # Remainder is 0
print(10 % 2) # Remainder is 0
```

Practical Uses:

- Checking Even/Odd: $\text{num \% 2} == 0$
- Clock Arithmetic: $13 \% 12 = 1$
- Finding the last digit: $123 \% 10 = 3$

POWER & FLOOR DIVISION

Power (**)

Exponentiation (raised to the power).

```
print(2 ** 5) # 2*2*2*2*2  
print(10 ** 3)
```

```
32  
1000
```

Floor Division (//)

Division that discards the decimal part.

```
print(5 / 2) # Float: 2.5  
print(5 // 2) # Floor: 2
```

```
2.5  
2
```

MATH OPERATORS SUMMARY

Symbol	Meaning	Example
<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>	Basic Math	Standard operations
<code>//</code>	Floor Division	<code>5 // 2 = 2</code>
<code>%</code>	Modulus	<code>5 % 2 = 1</code>
<code>**</code>	Power	<code>5 ** 2 = 25</code>

CASE STUDY: OHM'S LAW

```
v = float(input("Enter Voltage: "))
r = float(input("Enter Resistance: "))

current = v / r
power = v * current

print(f"Current: {current:.2f} A")
print(f"Power dissipated: {power:.2f} W")
```

PARALLEL RESISTANCE

Formula: $R_{eq} = \frac{R_1 \times R_2}{R_1 + R_2}$

```
r1 = float(input("R1 value: "))
r2 = float(input("R2 value: "))

req = (r1 * r2) / (r1 + r2)
print(f"Equivalent Resistance: {req:.2f} Ohms")
```

AUGMENTED ASSIGNMENT OPERATORS

Symbol	Meaning	Use	What it means
<code>+=</code>	Add and assign	<code>x += 3</code>	<code>x = x + 3</code>
<code>-=</code>	Subtract and assign	<code>x -= 3</code>	<code>x = x - 3</code>
<code>*=</code>	Multiply and assign	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code>	Divide and assign	<code>x /= 3</code>	<code>x = x / 3</code>
<code>//=</code>	Floor divide and assign	<code>x //= 3</code>	<code>x = x // 3</code>
<code>%=</code>	Modulus and assign	<code>x %= 3</code>	<code>x = x % 3</code>
<code>**=</code>	Power and assign	<code>x **= 3</code>	<code>x = x ** 3</code>

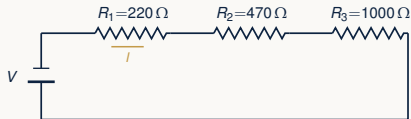
CASE STUDY: SERIES RESISTOR BANK

Add resistors one at a time, then apply a safety margin and convert units — all with augmented assignment.

```
total_r = 220 # R1
total_r += 470 # Add R2
total_r += 1000 # Add R3

total_r /= 1000 # Ohms to kilo-Ohms

print(f"Total: {total_r:.2f} kOhm")
```



Series circuit: $R_{total} = R_1 + R_2 + R_3$

COMMON MISTAKES

- ✗ `print(Hello)` – Missing quotes for text.
- ✗ `a = "10" + 5` – Cannot add string to integer.
- ✗ `x = input("v: ")` followed by `x + 1` – Forgetting to use `int()`.

LESSON SUMMARY

Display

Use `print()` with `sep` and `end`.

Storage

Variables hold `int`, `float`, and `str`.

Input

Always convert `input()` if doing math.

Math

Know your operators: `%`, `**`, `//`.

LAB EXERCISES

- 1. Power Bill Estimator:** Ask for appliance wattage, daily usage hours, and days in the month. Compute total energy in kWh ($\frac{W \times h \times d}{1000}$) and print the cost at Tk. 7.5/unit using an f-string with 2 decimal places.
- 2. Voltage Divider:** Given V_{in} , R_1 , and R_2 from user input, compute $V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$. Display both V_{out} and the voltage dropped across R_1 .
- 3. Battery Life:** Ask for battery capacity (mAh) and load current (mA). Compute runtime in hours using $/$, then convert the decimal hours into whole hours and remaining minutes using $//$ and $\%$.
- 4. Ohm's Challenge (Extended):** Take Voltage and Power as input. Calculate Resistance ($R = V^2 / P$) and Current ($I = P / V$), then print all three values aligned in one line using `sep` and f-string formatting.
- 5. Compound Interest Tracker:** Ask for principal, annual rate (%), and number of years. Using only `**` and `*` (no loops), compute the final amount with $A = P(1 + r)^n$ and the total interest earned.
- 6. Digital ID Generator:** Ask for a student's name and roll number. Build an ID string as first 3 letters of name (uppercase) + roll % 1000, using string slicing, `.upper()`, concatenation, and the modulus operator.

Q & A

Feel free to ask any questions!